

TECHNICAL REFERENCE

IncreMat

How the code, hardware, and platform fit together

A companion guide to the two Flutter apps — the **Caregiver** dashboard and the senior-facing **Play** app — plus the concepts they rely on: Dart & Flutter, Riverpod state management, Firebase, Bluetooth Low Energy (BLE), and NFC.

Sarcopenia care · Sit-to-stand training · Generated for the IncreMat team

Companion to the inline code comments in `src/incremat` and `src/incremat_play`

Contents

1. The big picture — what IncreMat is
2. Dart & Flutter fundamentals
3. State management with Riverpod
4. Firebase: Auth, Firestore & Messaging
5. How Bluetooth Low Energy (BLE) works
6. How NFC works
7. App architecture & folder map
8. Code walkthrough by layer
9. End-to-end: a rep's journey
10. Security model
11. Glossary

1 • The big picture — what IncreMat is

IncreMat is a smart exercise mat that turns the simple **sit-to-stand** motion into tracked, gamified daily training for older adults, to fight **sarcopenia** (age-related muscle loss).

There are three pieces of software in play, plus the physical mat:

Piece	Who uses it	What it does
The mat (ESP32-S3 firmware)	The senior sits on it	Senses each stand, counts reps, has an NFC reader, broadcasts over Bluetooth. (Firmware lives outside these repos.)
Caregiver app <code>src/incremat</code>	Family / carers	Dashboard: live reps, strength trends, mobility alerts, reports, managing seniors.
Play app <code>src/incremat_play</code>	The senior	Big, friendly, gamified: a virtual pet that grows as they exercise; reads reps live from the mat.

Why two apps? They are two *layers* of the same product. The Play app is the senior-facing motivation layer (large text, high contrast, a pet, sound). The Caregiver app is the data/dashboard layer for whoever supports them. They share the same Firebase backend, so what the senior does shows up on the carer's screen.

The clinical idea

Two well-known measures underpin the product:

- **30-Second Chair Stand Test (30CST)** — how many stands in 30 s; more = stronger legs. Used to set a starting daily goal and rate fall risk against age/sex norms (see `core/utils/chair_stand.dart`).
- **Five-Times Sit-to-Stand (5XSST)** — time to do 5 stands; slower over time signals declining mobility. The app captures an at-home proxy (time of the first 5 reps of a session) and watches its trend to raise *mobility alerts* (see `providers/mobility_alert_provider.dart`).

2 · Dart & Flutter fundamentals

Dart is the programming language; **Flutter** is the UI framework. One Dart codebase compiles to Android, iOS, web and desktop. The core idea of Flutter: *everything is a widget*, and the UI is a *function of state* — you describe what the screen should look like for the current data, and Flutter redraws when the data changes.

Dart syntax you'll see constantly

Syntax	Meaning
<code>final x = 5;</code>	A variable that can't be reassigned. <code>const</code> is similar but fixed at compile time.
<code>String?</code>	The <code>?</code> means <i>nullable</i> — it may hold <code>null</code> . Without it, null is impossible.
<code>x ?? y</code>	"x, or y if x is null." (Null-coalescing.) <code>x?.foo</code> calls foo only if x isn't null.
<code>a..b()..c()</code>	Cascade — call several methods on the same object.
<code>(int, String)</code>	A <i>record</i> — a lightweight group of values returned together.
<code>Future<T></code>	A value that will arrive later (async). You <code>await</code> it.
<code>Stream<T></code>	A sequence of values over time you can <code>listen</code> to (e.g. live reps).
<code>_name</code>	A leading underscore = <i>private</i> to its file.
<code>=></code>	Shorthand for a function that returns one expression.

async / await

Anything that takes time (network, Bluetooth, disk) returns a `Future`. Marking a function `async` lets you `await` each step and write it top-to-bottom as if it were sequential:

```
Future<void> connect() async {
  await turnOnBluetooth(); // pause here until done
  final mat = await scan(); // then this
  await mat.connect();     // then this
}
```

Widgets: Stateless vs Stateful

- **StatelessWidget** — draws purely from its inputs; never changes itself (e.g. a label).
- **StatefulWidget** — owns mutable state and rebuilds when it calls `setState` (e.g. a form, a toggle).
- In this app you'll mostly see **ConsumerWidget** / **ConsumerStatefulWidget** — the same thing but with access to Riverpod (next section).

The build method is the heart of every widget: it returns the widget tree to display. Flutter calls it again whenever the widget's data changes, and efficiently updates only what differs. You'll see deeply nested widgets like `Scaffold > SafeArea > Column > ...` — this *composition* (small widgets inside bigger ones) is how all Flutter UIs are built.

3 · State management with Riverpod

As apps grow, you need a tidy way to share data (the logged-in user, the list of seniors, live reps) across many screens without passing it down by hand. **Riverpod** is the library IncreMat uses for this. A **provider** is a named, globally-readable piece of state or logic.

Reading providers

Call	Use it for
<code>ref.watch(p)</code>	Read a value AND rebuild this widget whenever it changes. The default for UI.
<code>ref.read(p)</code>	Read once, no rebuild. For one-off actions (e.g. inside a button press).
<code>ref.listen(p, cb)</code>	Run a side effect on change (navigate, show a toast) without rebuilding.

Kinds of provider used in IncreMat

Type	Holds	Example in the code
Provider	A computed value / service	<code>hardwareServiceProvider</code>
<code>StreamProvider</code>	A live stream as loading/data/error	<code>authStateProvider</code> , <code>seniorsStreamProvider</code>
<code>StateProvider</code>	One simple mutable value	<code>activeExerciserIdProvider</code>
<code>NotifierProvider</code>	A class with state + methods	<code>liveSessionProvider</code> , <code>localeProvider</code>
<code>AsyncNotifierProvider</code>	Async state (with loading/error)	<code>authNotifierProvider</code>
<code>*.family</code>	A provider parameterised by an argument	<code>sessionRepositoryProvider(seniorId)</code>

AsyncValue — anything async (a stream/future) arrives wrapped as an `AsyncValue` with three states: *loading*, *data*, *error*. The UI handles all three at once with `.when(data:..., loading:..., error:...)`, so loading spinners and error messages fall out naturally. See [features/auth/auth_wrapper.dart](#).

```
// A provider:
final localeProvider = NotifierProvider<LocaleNotifier, Locale>(LocaleNotifier.new);

// A widget reading it – rebuilds when the language changes:
class MyApp extends ConsumerWidget {
  Widget build(context, ref) {
    final locale = ref.watch(localeProvider);
    return MaterialApp(locale: locale, ...);
  }
}
```

Every app is wrapped in a `ProviderScope` (in `main.dart`) — that's the container that stores all provider state.

4 · Firebase: Auth, Firestore & Messaging

Firebase is Google's backend-as-a-service. IncreMat uses three parts of it, so there's no custom server to run.

Authentication

- **Caregiver app** — real email/password accounts (`auth_provider.dart` , `login_screen.dart`).
- **Play app** — seniors don't manage passwords. They sign in with a *join code*; under the hood the app uses Firebase **anonymous auth** so database reads are still authenticated (`services/auth_service.dart`).

Cloud Firestore (the database)

Firestore is a cloud **NoSQL** database organised as **collections** (folders) of **documents** (JSON-like records), which can themselves contain sub-collections. IncreMat's shape:

```
seniors/{seniorId}      ← a senior's profile
├── sessions/{autoId}   ← each completed exercise session
├── live/current         ← mirror of the in-progress session (live)
├── caregivers/{caregiverId} ← who may view this senior
├── pets/{petId}        ← Play app: the senior's pets
└── expEvents/{id}      ← Play app: milestone history
caregiver_access/{caregiverId} ← list of seniorIds a carer can see + FCM tokens
nfc_uids/{uidHex}       ← card UID → seniorId
joinCodes/{CODE}        ← join code → seniorId
duets/{code}            ← Play app: a paired exercise lobby
```

Two ways to read. `.get()` reads once (a snapshot in time — used for reports). `.snapshots()` returns a *live stream* that re-emits whenever the data changes — this is why the caregiver's dashboard updates the instant the senior does a rep. Most repositories expose both.

Offline persistence. Firestore caches data on the device and *queues writes* when offline, replaying them when the connection returns. That's why a logged session is never lost even with no signal — and why the session-save code uses a `.timeout()`: the write is safely queued even if the server acknowledgement is slow.

The model ↔ Firestore pattern

Every data type has a class with three companions you'll see everywhere (e.g. `models/senior.dart`):

- `fromMap(map, id)` — build the object *from* a Firestore document (with defensive casts & defaults).
- `toMap()` — turn the object *into* a document to save.
- `copyWith(...)` — make a modified copy (because the fields are immutable `final`).

Cloud Messaging (FCM)

Push notifications for decline/fitness reminders even when the app is closed. The device registers a *token* (its push address) stored on the caregiver's access doc (`services/notifications/push_service.dart`). Day-to-

day reminders are *local* notifications fired by the phone itself (`notification_service.dart`).

5 • How Bluetooth Low Energy (BLE) works

BLE is how the phone talks to the mat wirelessly, using very little power. Understanding five words unlocks the whole `ble_hardware_service.dart` / `mat_ble_service.dart` files.

Term	Meaning
Advertise	A BLE device broadcasts its name so phones can find it. The mat advertises as <code>"IncreMat"</code> .
Scan	The phone listens for advertising devices.
GATT	The data layout a device exposes once connected — a table of services & characteristics.
Service	A logical group of related values, named by a UUID.
Characteristic	An individual value you can <i>read</i> , <i>write</i> , or <i>subscribe</i> to. Also UUID-named.
Notify	A subscription: the device <i>pushes</i> a new value whenever it changes — no polling.
UUID	A 128-bit id. Phone & firmware must agree on the exact same UUIDs — see <code>ble_constants.dart</code> .
RSSI	Signal strength in dBm — negative; closer to 0 = stronger (−45 great, −90 weak).

The connection sequence (in `connect()`)

1. **Turn on** the Bluetooth radio if needed.
2. **Scan** for a device advertising the name "IncreMat" (give up after 15 s).
3. **Connect** to it (fail after 10 s if unresponsive).
4. **Discover** its GATT table; find our service by UUID.
5. **Subscribe** (`setNotifyValue(true)`) to each characteristic we care about and attach a handler.
6. Each handler **decodes the bytes** and pushes a value onto a Dart `Stream` the app watches.

The mat's characteristics

Characteristic	Mode	Bytes → meaning
repCount	notify	2 bytes, little-endian uint16 = cumulative reps
repSpeed	notify	4 bytes, little-endian float32 = avg seconds/rep
battery	notify	1 byte = battery %
matPlaced	notify	1 byte: 1 = on chair, 0 = removed
nfcScan	notify	text bytes = tapped card's UID
musicTrack	write	text bytes = track name to sync

Why the bit-shifting? BLE delivers raw bytes, so the code must rebuild numbers itself. A rep count arrives as two bytes, low first ("little-endian"): `count = data[0] | (data[1] << 8)` — keep the low byte, shift the high byte up by 8 bits ($\times 256$), OR them. e.g. bytes `[44, 1]` $\rightarrow 44 | 256 \rightarrow 300$.

Two connections at once. The senior's Play app AND the caregiver app can both read the mat live, so the firmware is configured to allow **two simultaneous BLE connections** (and to keep advertising after the first connects). The Play app reads reps straight off the mat for an instant on-screen count — no Firebase round-trip.

6 • How NFC works

NFC (Near-Field Communication) lets the phone or the mat read a card held within a couple of centimetres. IncreMat uses it to answer one question: *"who is on the mat right now?"*

The key concept: the UID

Every NFC chip has a permanent hardware **UID** (unique id). IncreMat never writes data to cards — it only reads this UID and looks it up. Any NFC card works: EZ-Link, building passes, bank cards, blank NTAG stickers. The UID bytes are formatted as a lowercase hex string, e.g. `04ab10c3` (see `bytesToHex` in `services/nfc/nfc_service.dart`).

NFC is several radio standards, not one. Cards speak NfcA, NfcB/IsoDep, NfcF or NfcV. `extractUid()` simply tries each "technology view" until one returns an identifier — which is why it works with such a wide range of cards.

Two readers, same result

- **Phone NFC** — tap a card on the phone; handled by the `nfc_manager` plugin.
- **Mat NFC** — the mat's own reader sends the UID to the phone over BLE (the `nfcScan` characteristic above).

Either way, the UID is looked up in `nfc_uids/{uid}` → a `seniorId` (`nfc_uid_service.dart`). The app then attributes the session to that senior. This is what lets several seniors share one mat: tap your card, and your reps are logged to *you*.

Security: a card UID is treated as a credential — knowing it lets you identify as that senior — so it's only ever used after a caregiver has explicitly enrolled the card (`nfc_write_sheet.dart`). It's never shared or used for pairing.

7 • App architecture & folder map

Both apps follow the same **layered** structure. Data flows up; actions flow down. Each layer only knows about the one below it, which keeps things testable and easy to change.

```

UI (features/)      screens & widgets – what the user sees
  | watches providers / calls their methods
State (providers/)  Riverpod – holds & combines app state
  | calls
Services (services/) talk to the outside world: BLE, NFC, Firestore, audio
  | read/write
Models (models/)    plain data classes (Senior, SessionLog, Pet, ...)
Core (core/)         constants, theme, utilities (colors, rep_segmenter, ...)
  
```

Caregiver app — `src/incremat/lib` (56 files)

Folder	What lives there
<code>core/</code>	Brand colours, text styles, theme, BLE UUIDs, the chair-stand norms and the RepSegmenter .
<code>models/</code>	<code>Senior</code> , <code>SessionLog</code> , <code>SyncStatus</code> .
<code>services/hardware/</code>	The <code>HardwareService</code> interface + real BLE, simulator & mock implementations.
<code>services/</code> (rest)	NFC, Firestore repositories (seniors, sessions, join codes), notifications, sync.
<code>providers/</code>	Auth, hardware, seniors, live session, insights, mobility alerts, music, etc.
<code>features/</code>	Screens grouped by area: auth, home, insights, hardware, seniors, settings, reports.

Play app — `src/incremat_play/lib` (33 files)

Folder	What lives there
<code>core/</code>	Vivid palette (+ dark / high-contrast sets), Nunito typography, theme, BLE UUIDs.
<code>models/</code>	<code>Pet</code> & species/stages, <code>Senior</code> , <code>LiveSession</code> , <code>DuetSession</code> .
<code>services/</code>	Direct-from-mat BLE, gamification (game / pet / consistency), senior & auth services.
<code>providers/</code>	Auth, mat connection, senior/pet state, accessibility, onboarding, locale.
<code>features/</code>	Home, login, sanctuary, duet, and the celebration screens (egg award, hatching, evolution).

The swappable-hardware trick (caregiver app). The app depends only on the `HardwareService` interface, never a concrete class. `hardwareServiceProvider` hands it either the real `BleHardwareService` or the fake `SimulatorHardwareService` based on a toggle — so the whole app can be demoed without a physical mat, and the UI never knows the difference. This is "dependency inversion."

8 • Code walkthrough by layer

Core — the foundations

- `app_colors.dart` / `app_text_styles.dart` / `app_theme.dart` — the single source of truth for the look. Defined once, used everywhere.
- `ble_constants.dart` — the shared UUID "contract" with the firmware.
- `chair_stand.dart` — clinical norms for the 30CST and the recommended-goal maths.
- `rep_segmenter.dart` — **important**. The mat reports one ever-growing total; this turns it into per-session reps by tracking a baseline, and handles the edge cases (reconnects, mat resets, user switches).
Session reps = current total – baseline.

Services — the outside world

- `ble_hardware_service.dart` — the real Bluetooth driver (scan → connect → discover → notify → decode → stream).
- `simulator_hardware_service.dart` — a fake mat driven from the Developer screen; same interface.
- `session_repository.dart` / `senior_repository.dart` — Firestore read/write, hidden behind plain methods. Note `watch...` (live streams) vs `get...` (one-shot).
- `join_code_service.dart` — generates collision-free join codes with a secure RNG and a Firestore transaction.
- `nfc_service.dart` / `nfc_uid_service.dart` — read a card UID, map it to a senior.

Providers — the brain

- `live_session_provider.dart` — listens to the mat's rep stream, accumulates the current session via the `RepSegmenter`, attributes it to the right senior, and saves it to Firestore. Holds the `finalizeCurrentSession()` logic that guarantees a session is written before the active exerciser switches.
- `senior_provider.dart` — the central hub: who's selected, who's on the mat, the session streams. Returns mock data when logged out so demo mode "just works."
- `insights_provider.dart` — turns raw sessions into the dashboard numbers (today, weekly bars, consistency, monthly totals, latest 5-rep time), folding in any live in-progress session.
- `mobility_alert_provider.dart` — the clinical early-warning: a pure function analyses 28 days of 5-rep times and flags a sharp (day) or sustained (week) slowdown, then notifies the carer once per event.
- `session_music_provider.dart` — the layered-stem music engine: instruments fade in as reps climb, with a spoken count every 5 reps (the music ducks under the voice).

Play app — the gamification

- `models/pet.dart` — pets, species, stages (egg→baby→young→adult), EXP and history events.
- `consistency_service.dart` — the rules: what an ISO week is, and when an egg is earned.
- `game_service.dart` — orchestrates the reward after each session (award egg? add EXP? evolve?).
- `mat_ble_service.dart` — a focused BLE client that reads reps straight off the mat as a small state machine (disconnected→scanning→connecting→connected).

9 • End-to-end: a rep's journey

Tying it all together — what happens when a senior stands up once:

1. The mat's sensor detects the stand and increments its internal counter.
2. It pushes the new total over BLE on the `repCount` characteristic (notify).
3. `BleHardwareService` decodes the 2 bytes and emits the number on `repCountStream`.
4. `LiveSessionNotifier` hears it, runs it through the `RepSegmenter` to get this session's rep count, and attributes it to the active senior.
5. It mirrors the running count to `seniors/{id}/live/current` in Firestore.
6. The **caregiver dashboard** watches that doc and the insights provider, so the number ticks up live. The **Play app** shows it even faster — read straight off the mat over its own BLE link.
7. When the session ends (idle, app backgrounded, or an NFC user-switch), it's written permanently to `seniors/{id}/sessions`. The Play app's `GameService` may then award an egg or evolve the pet.

10 • Security model

Credentials are never shared or reused for pairing.

- **Join code** ([ROSE-4821](#)) — a login-style credential to link a caregiver to a senior. Generated with a cryptographic RNG; unique; used only for that linking.
- **NFC UID** — identifies a senior on the mat; only used after a caregiver enrolls the card.
- **Duet code** — the head-to-head/co-op pairing code is *ephemeral* and contains no login credential, so it's safe to share to pair up. It is deliberately *not* the join code.

Sensitive account actions (e.g. deleting an account) require recent re-authentication, and Firestore security rules — not the app — are the real gate on who can read/write each document.

11 • Glossary

Term	Meaning
Sarcopenia	Age-related loss of muscle mass & strength — the condition IncreMat targets.
30CST	30-Second Chair Stand Test — stands in 30 s; more = stronger.
5XSST	Five-Times Sit-to-Stand — time for 5 stands; slower over time = declining mobility.
Widget	The building block of a Flutter UI.
Provider	A Riverpod unit of shared state/logic.
Future / Stream	A value that arrives later / a sequence of values over time.
Firestore	Google's cloud NoSQL database (collections of documents).
BLE	Bluetooth Low Energy — low-power wireless link to the mat.
GATT / Characteristic	The data layout a BLE device exposes / one readable-or-writable value in it.
Notify	A BLE subscription where the device pushes new values automatically.
RSSI	Signal strength in dBm (negative; nearer 0 is stronger).
Little-endian	Byte order where the least-significant byte comes first.
NFC / UID	Tap-to-read radio / a card's permanent unique id.
FCM	Firebase Cloud Messaging — push notifications.
Repository	A class that hides database details behind plain methods.
RepSegmenter	Converts the mat's running total into per-session reps.

This reference is a companion to the inline comments now present in every file under `src/incremat/lib` and `src/incremat_play/lib`. Read a screen's comments alongside the relevant section here for the fullest picture.